

Tâche 2 — IFT3913 (GraphHopper)

Binôme :

- Léa Hemidj
- Ariane Lavarrière

1) Classes ciblées

Trois classes déjà testées mais non couvertes à 100 % :

- `com.graphhopper.util.ArrayUtil`
- `com.graphhopper.util.DistanceCalcEarth`
- `com.graphhopper.coll.MinHeapWithUpdate`

Objectif : ajouter des cas bords/propriétés/erreurs (dont 3D/IDL) pour augmenter la couverture et la détection de mutants.

2) Nouveaux tests ajoutés (10 fichiers)

Package util :

- `ArrayUtilCoverageTest.java`
- `ArrayUtilExtraTest.java`
- `ArrayUtilFakerTest.java` (utilise java-faker)
- `ArrayUtilMutantsTest.java`
- `DistanceCalcEarthCoverageTest.java`
- `DistanceCalcEarthExtraTest.java`
- `DistanceCalcEarthMutantsTest.java`

Package coll :

- `MinHeapWithUpdateCoverageTest.java`
- `MinHeapWithUpdateExtraTest.java`
- `MinHeapWithUpdateMutantsTest.java`

3) Documentation détaillée des cas

A) `ArrayUtil`

`calcSortOrder_stableByPrimaryThenSecondaryAscending`

Intention : vérifier l'ordre stable par clé primaire puis secondaire (tri ascendant) avec la surcharge `int[] + n`.

Données : primary={2,2,2,3,1,1}, secondary={5,1,7,0,9,8}, n=primary.length.

Oracle : indices triés = {5,4,1,0,2,3} (les 1 avant les 2, et départage par secondary).

calcSortOrder_withN_usesPrefixOnly

Intention : la variante avec n ne trie que le préfixe (pas d'effet sur la queue).

Données : primary={9,1,1,0,5}, secondary={0,7,3,4,2}, n=3.

Oracle : ordre sur le préfixe {2,1,0}.

applyOrder_roundtripWithCalcSortOrder

Intention : cohérence calcSortOrder → applyOrder.

Données : arr={40,10,20,30}, keys={4,1,2,3}.

Oracle : applyOrder(arr, order) renvoie {10,20,30,40}.

applyOrder_throwsWhenOrderLongerThanArray

Intention : valider la vérification d'entrées invalides.

Données : arr={10,20}, order={0,1,2} (plus long que arr).

Oracle : lève IllegalArgumentException.

invert_IntArrayList_variant

Intention : tester la variante HPPC IntArrayList.

Données : permutation p=[2,0,3,1].

Oracle : inverse inv=[1,3,0,2] (car inv[p[i]]=i).

permutation_isPermutation_onRandomSizeFromFaker (Faker)

Intention : couvrir des chemins dépendant de la taille n via une valeur pseudo-réaliste variable (8–20), tout en gardant la permutation déterministe (seed).

Données : n = Faker().number().numberBetween(8,20) ; perm = ArrayUtil.permutation(n, new Random(42)).

Oracle : perm.size()==n et ArrayUtil.isPermutation(perm) renvoie true.

invert_roundtrip_onPermutation

Intention : propriété fondamentale inv[p[i]]=i.

Données : p = permutation(20, seed=7).

Oracle : pour tout i , $\text{inv.get(p.get(i))} == i$.

subList_edges_emptyAndFull

Intention : cas bords sous-liste (vide et totalité).

Données : base [3,1,4,1,5], sous-liste [2,2] (vide) et [0, size] (pleine).

Oracle : égalités exactes avec `HPPC IntArrayList.from(...)`.

merge_allDuplicatesRemoved

Intention : fusion de deux tableaux triés avec déduplication.

Données : $a=\{1,1,2,2,3,3\}$, $b=\{1,2,3,3,3,4\}$.

Oracle : $\{1,2,3,4\}$.

B) DistanceCalcEarth

edgeDistance_clampsBeforeA_equalsDistToA

Intention : projection avant $A \rightarrow$ clamp sur A.

Données : segment $A(0,0) \rightarrow B(0,1)$; point $R(-0.1,0.0)$ avant A.

Oracle : $\text{edgeDist}(R,AB) == \text{dist}(R,A)$; $\text{validEdgeDistance}(...) == \text{false}$.

edgeDistance_clampsAfterB_equalsDistToB

Intention : projection après $B \rightarrow$ clamp sur B.

Données : même segment ; point $R(0.1,1.0)$ après B.

Oracle : $\text{edgeDist}(R,AB) == \text{dist}(R,B)$; $\text{validEdgeDistance}(...) == \text{false}$.

createBBox_negativeRadius_throws

Intention : rayon ≤ 0 invalide.

Données : rayons -0.1 et 0.0.

Oracle : `IllegalArgumentException`.

isDateLineCrossOver_edges

Intention : vrai si écart $> 180^\circ$, faux à 180° (cas limite).

Données : couples $(179.9 \leftrightarrow -179.9)$, $(170 \leftrightarrow -170)$, $(-180 \leftrightarrow 180) \Rightarrow \text{true}$; $(0 \leftrightarrow \pm 180) \Rightarrow \text{false}$.

Oracle : booléens attendus.

`projectCoordinate_normalizesLongitude_acrossIDL`

Intention : longitudes normalisées dans $[-180,180]$ lors d'un passage de l'IDL.

Données : `project(0, 179, 400000m, 90°)`.

Oracle : $\text{lon} \in [-180,180]$ et < 0 (on "passe" côté Ouest).

`calcDistance_overPointList_2Dvs3D`

Intention : distance $3D \geq 2D$.

Données : 3 points 2D vs mêmes points avec altitudes (0-50-0).

Oracle : $d3 \geq d2$ et $d2, d3 > 0$.

`denormalize_normalize_roundtrip_smallDistance`

Intention : round-trip dist $\leftrightarrow$ normalized.

Données : `dist=37.5m`.

Oracle : tolérance $1e-6$ sur le retour.

`calcNormalizedDist_symmetry_and_zero`

Intention : symétrie $A \leftrightarrow B$ et distance nulle à soi-même.

Données : $(\text{Paris} \leftrightarrow \text{Berlin}) + (\text{Paris} \leftrightarrow \text{Paris})$.

Oracle : $d(A,B) == d(B,A)$; $d(A,A) == 0$.

`validEdgeDistance_borderline_zeroDotProducts_false`

Intention : cas frontières où $(AR \cdot AB) == 0$ ou $(RB \cdot AB) == 0 \Rightarrow$ hors segment.

Données : segment $A(0,0) \rightarrow B(1,0)$, points $\sim$ alignés sur A et B.

Oracle : false dans les deux cas.

`calcNormalizedEdgeDistance3D_handlesNaNAndFactorNaNPath`

Intention : couvrir (a) $\text{norm} == 0 \Rightarrow$ factor NaN corrigé à 0, (b) altitude NaN $\Rightarrow$ repli 2D.

Données : $R=A=B$ (mêmes coords, $z=5$) pour (a) ; $z=\text{NaN}$ côté R pour (b).

Oracle : (a) distance dénormalisée ≈ 0 ; (b) égalité exacte avec la 2D.

projectCoordinate_headingWrapsCorrectly

Intention : $360^\circ \equiv 0^\circ$.

Données : project(0,0,1000,360) vs project(0,0,1000,0).

Oracle : lat/lon identiques (tolérance 1e-9).

C) MinHeapWithUpdate

update_decreaseKey_movesUp

Intention : decrease-key fait remonter l'élément (devient min).

Données : push (2,20f), (1,10f), (3,30f), puis update(2,1f).

Oracle : peekId()==2.

clear_thenReuse_isClean

Intention : clear() réinitialise complètement (réutilisation d'ID sans fuite).

Données : push deux éléments, clear(), puis repush id=1.

Oracle : isEmpty() après clear(), puis peekId()==1.

update_onMissingId_throws

Intention : update sur id absent lève IllegalStateException.

Données : tas vide, update(1, 1.0f).

Oracle : exception attendue.

update_decreaseKey_bubblesUpToTop

Intention : decrease-key remonte jusqu'au sommet + cohérence poll().

Données : push (2,5f), (4,3f), (1,7f), update(1,2.5f).

Oracle : peekId()==1, puis poll()==1.

update_increaseKey_movesDown

Intention : increase-key fait descendre l'élément ; ordre de poll() valide.

Données : push (0,1f),(1,2f),(2,3f), update(0,10f).

Oracle : peekId()==1, poll()==1, puis 2, puis 0.

peekValue_consistentAfterUpdates

Intention : peekValue() reflète bien la dernière mise à jour.

Données : push (2,5f),(1,8f), update(1,1f).

Oracle : peekValue()==1f, peekId()==1.

4) Java-Faker — ajout et justification

Dépendance (scope test) : com.github.javafaker:javafaker:1.0.2

Justification :

On fait varier réalistement la taille n dans [8,20], ce qui force des chemins différents (allocation, boucles, logique d'indices) qu'un seul cas statique ne couvrirait pas.

On garde les tests déterministes : la permutation est générée avec new Random(42).

Le Faker est isolé aux tests (scope test), aucune incidence production.

5) PIT — configuration & commandes (depuis core/)

Java 17

export JAVA_HOME=\$(/usr/libexec/java_home -v 17)

export PATH="\$JAVA_HOME/bin:\$PATH"

Maven (si non installé) : brew install maven

mvn -v

A. Exécuter uniquement nos tests

Avec le profil tache2

mvn -pl core -Ptache2 clean test -Dsurefire.printSummary=true

Résultat attendu : Tests run: 26, Failures: 0, Errors: 0, Skipped: 0

Sans profil (équivalent, selon vos préférences)

mvn -pl core -Dtest=*ExtraTest,*MutantsTest,*CoverageTest,ArrayUtilFakerTest clean test -Dsurefire.printSummary=true

Remarque : nous n'exécutons pas les tests originaux du projet comme UnzipperTest

B. Rapport PIT (mutation testing)

La config PIT (JUnit5, cibles, exclusions dont UnzipperTest) est déjà dans core/pom.xml.

Donc aucun paramètre additionnel n'est requis.

Générer le rapport

repartir propre

```
rm -rf core/target/pit-reports
```

lancer PIT sur les cibles définies dans le POM

```
mvn -pl core org.pitest:pitest-maven:mutationCoverage
```

ouvrir le rapport

```
open core/target/pit-reports/index.html
```

Option : cibler uniquement le package util

```
mvn -pl core org.pitest:pitest-maven:mutationCoverage \
```

```
-DtargetClasses='com.graphhopper.util.*' \
```

```
-DtargetTests='com.graphhopper.util.*'
```

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
3	90% 311/347	81% 304/374	90% 304/338

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.graphhopper.coll	1	99% 88/89	83% 43/52	83% 43/52
com.graphhopper.util	2	86% 223/258	81% 261/322	91% 261/286

Report generated by [PIT](#) 1.20.6

Enhanced functionality available at [arcmutate.com](#)

PIT APRÈS (avec nos tests ; UnzipperTest exclu)

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
3	97% 337/347	89% 334/374	91% 334/366

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.graphhopper.coll	1	100% 89/89	83% 43/52	83% 43/52
com.graphhopper.util	2	96% 248/258	90% 291/322	93% 291/314

Report generated by [PIT](#) 1.20.6

Enhanced functionality available at [arc42.org](#)

Rapports générés

Avant : `target/pit-report-before/index.html`

Après : `target/pit-report/index.html`

6) Résultats PIT (avant → après)

- Line Coverage (mutées) : 311/347 (90 %) → 337/347 (97 %)
- Mutation Coverage : 304/374 (81 %) → 334/374 (89 %)
- Test Strength : 90 % → 91 %

Extraits : coll ≈ 83 % (mutations) avec lignes 88/89 → 89/89 (100 %) ; util 81 % → 90 % (hausse sur MathMutator et NegateConditionals).

7) Mutants nouvellement tués — analyse

MathMutator (DistanceCalcEarth)

Les tests `calcNormalizedEdgeDistance3D_handlesNaNAndFactorNaNPath` (`norm==0` ⇒ `factor NaN` corrigé à 0) et `projectCoordinate_normalizesLongitude_acrossIDL` forcent des chemins où de petites altérations arithmétiques (`+/-/*//`) changent le résultat (comparaisons 2D vs 3D et normalisation de longitude). Les assertions d'égalité/ordres tuent ces mutants.

NegateConditionals / ConditionalsBoundary (DistanceCalcEarth)

`edgeDistance_clampsBeforeA_equalsDistToA`, `edgeDistance_clampsAfterB_equalsDistToB` et `validEdgeDistance_borderline_zeroDotProducts_false` couvrent des branches limites (avant A, après B, produits scalaires ≈ 0). Inverser `<=`/`<` ou négation du prédicat change la projection/clamp et casse nos égalités — mutants tués.

PrimitiveReturns / VoidMethodCall (MinHeapWithUpdate)

update_decreaseKey_movesUp, update_increaseKey_movesDown, clear_thenReuse_isClean, update_onMissingId_throws imposent des invariants forts (peekId, ordre de poll, exceptions). Supprimer un appel interne ou retourner une valeur par défaut brise immédiatement les assertions — mutants tués.

IncrementsMutator (ArrayUtil)

invert_roundtrip_onPermutation vérifie $\text{inv}[p[i]] = i$ pour tout i . Un $i++/--$ muté ou une variation d'indexation fausse la relation — mutant tué.

Null/BooleanReturnVals

Les tests d'exception (applyOrder_throwsWhenOrderLongerThanArray, update_onMissingId_throws) et les invariants (isPermutation) capturent des retours null/true/false mutés — mutants tués.

8) Fichiers concernés (noms)

- ArrayUtilCoverageTest.java
- ArrayUtilExtraTest.java
- ArrayUtilFakerTest.java
- ArrayUtilMutantsTest.java
- DistanceCalcEarthCoverageTest.java
- DistanceCalcEarthExtraTest.java
- DistanceCalcEarthMutantsTest.java
- MinHeapWithUpdateCoverageTest.java
- MinHeapWithUpdateExtraTest.java
- MinHeapWithUpdateMutantsTest.java
- pom.xml (core, avec java-faker + plugin PIT)